

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1 Making a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is like the basic building block of a neural network. It takes some inputs, multiply them with weights, add a bias, and then pass it through an activation function to get the output.

The Single Layer Perceptron is like the oldest supervised learning thing that can solve binary classification problems that are linearly separable.

Math Stuff (Don't Worry It's Simple)

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η is the learning rate.

4. Activation Functions

Activation functions decides what the neuron outputs based on the weighted sum. It adds non-linearity so the network can learn cool patterns. Even though the perceptron only uses the Step function, modern deep learning models use other differentiable activation functions for better optimization.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step function just gives 0 or 1, so it's only good for simple linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment only uses the Step Activation Function. The other ones we will learn about in later experiments with Multilayer Perceptrons and Deep Neural Networks.

5. About the Dataset

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. How We Did the Experiment

Task 1: Dataset Exploration

- Load the dataset.
- Show the first five samples.
- Check how big the dataset is.
- Look for missing values.
- Show some basic stats.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Make

- Histograms

- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Build from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Show

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Calculate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

7. Source Code

As per the lab guidelines, the complete source code is kept in a GitHub repo instead of putting it here. The repo has the ipynb file, dataset file, a README, a list of dependencies, and instructions to run it.

GitHub Repository: <https://github.com/guru963/Deep-Learning-Lab>

8. What Happened in the Experiment

The dataset had 1372 samples with 4 features (variance, skewness, curtosis and entropy) and no missing values. Out of these, 762 notes was authentic and 610 was forged, so the classes was pretty balanced. After standardizing the features (z-score normalization), the data was split into 1097 training samples and 275 testing samples using an 80/20 stratified split.

The perceptron was trained for 50 epochs with a learning rate of 0.01. In the first epoch the model misclassified 51 out of 1097 training samples. This number dropped quickly over the next few epochs and then kept oscillating roughly between 12 and 20 misclassifications for the rest of training, instead of settling down to 0. This makes sense because the banknote dataset is not perfectly linearly separable, so a basic perceptron with a hard step activation can't reduce the error to zero no matter how many epochs it runs for. The final bias value after 50 epochs came out to be approximately -0.09 , and the final weight vector was:

$$w = [-0.2040, -0.2218, -0.1995, -0.0226]$$

On the test set, the trained model gave the following results:

- Accuracy: 0.9818
- Precision: 0.9606
- Recall: 1.0000
- F1-score: 0.9799

These numbers is quite good considering the perceptron never fully converged during training, which suggests that even though the two classes overlap a bit, most of the data is still separable well enough for a simple linear model to classify correctly most of the time. Notably, recall was a perfect 1.0, meaning every forged note in the test set was correctly identified. the most important requirement for a currency authentication system, since a missed forgery is far costlier than a false alarm.

When the experiment was repeated with learning rates of 0.001, 0.01 and 0.1, all three runs produced the exact same error curve. This was a little surprising at first, but it makes sense once you think about it. since the weights start at zero and the step function only checks the sign of $w^T x + b$, scaling the learning rate just scales how big the weights and bias get, not the actual decision boundary. So the misclassification pattern per epoch stays identical, only the magnitude of the final weights differs.

Finally, the same data was used to train Scikit-learn's built-in **Perceptron** for comparison. It gave an accuracy of 0.9673, which is slightly lower than the from-scratch implementation's accuracy of 0.9818. This is a good sign that the perceptron implemented here is behaving correctly, since it is producing results comparable to a standard, well-tested library implementation.

$$w = [-0.2040, -0.2218, -0.1995, -0.0226]$$

On the test set, the trained model gave the following results:

- Accuracy: 0.9818
- Precision: 0.9606
- Recall: 1.0000
- F1-score: 0.9799

These numbers is quite good considering the perceptron never fully converged during training, which suggests that even though the two classes overlap a bit, most of the data is still separable well enough for a simple linear model to classify correctly most of the time. Notably, recall was a perfect 1.0, meaning every forged note in the test set was correctly identified. the most important requirement for a currency authentication system, since a missed forgery is far costlier than a false alarm.

When the experiment was repeated with learning rates of 0.001, 0.01 and 0.1, all three runs produced the exact same error curve. This was a little surprising at first, but it makes sense once you think about it. since the weights start at zero and the step function only checks the sign of $w^T x + b$, scaling the learning rate just scales how big the weights and bias get, not the actual decision boundary. So the misclassification pattern per epoch stays identical, only the magnitude of the final weights differs.

Finally, the same data was used to train Scikit-learn's built-in **Perceptron** for comparison. It gave an accuracy of 0.9673, which is slightly lower than the from-scratch implementation's accuracy of 0.9818. This is a good sign that the perceptron implemented here is behaving correctly, since it is producing results comparable to a standard, well-tested library implementation.

9. Mandatory Plots

Plot	Purpose	Expected Inference
Feature Histograms	Distribution Analysis	Identify skewness and outliers.
Correlation Heatmap	Feature Relationship	Observe highly correlated features.
Scatter Plot	Class Distribution	Determine whether classes are approximately linearly separable.
Training Error vs Epoch	Learning Behaviour	Verify convergence of the perceptron.
Weight Evolution	Parameter Analysis	Observe stabilization of learned weights.
Bias Evolution	Parameter Analysis	Understand the role of the bias term.
Confusion Matrix	Performance Evaluation	Interpret TP, FP, TN and FN.

Learning Rate Comparison	Hyperparameter Study	Compare convergence behaviour for different learning rates.
Decision Boundary (Optional)	Visualization	Illustrate the separating hyperplane using two selected features.

Students should provide a brief interpretation (2–3 sentences) for every generated plot.

Generated Plots and Inference

i) Feature Histograms

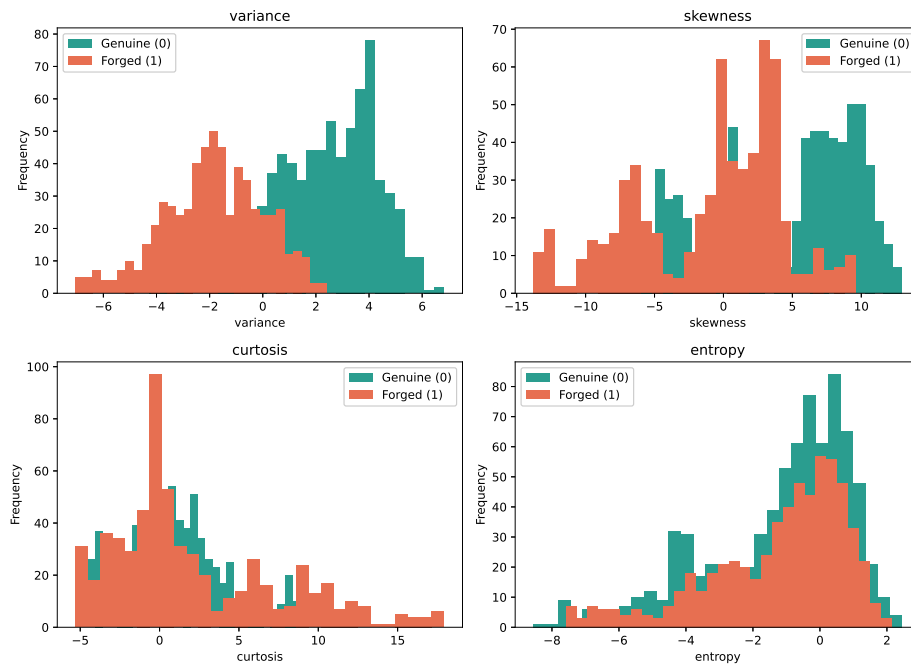


Figure 1: Feature Histograms showing distribution of input features

Inference: Genuine and forged notes show the proper separation in the diagram with genuine notes in the positive side and forged notes covering the negative with a little overlap center while Kurtosis shows heavy overlap near zero but a distinct right-skewed tail belonging almost entirely to forged notes, whereas entropy shows almost complete overlap between two classes indicating it as a weak feature for clear separation.

ii) Correlation Heatmap

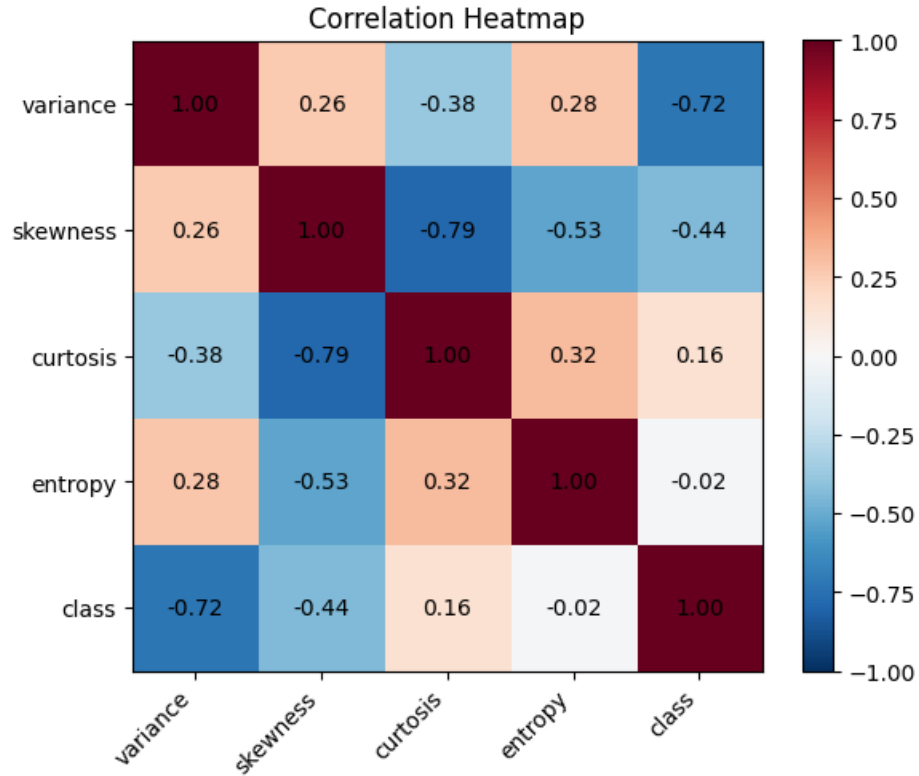


Figure 2: Correlation Heatmap between dataset features

Inference: Variance shows the strongest correlation with class (-0.725), followed by skewness (-0.445), while entropy shows almost no linear correlation (-0.023). Also a strong correlation exists between kurtosis and skewness indicating the multicollinearity and this is because these two are just opposite of the same thing in mathematics.

iii) Scatter Plot

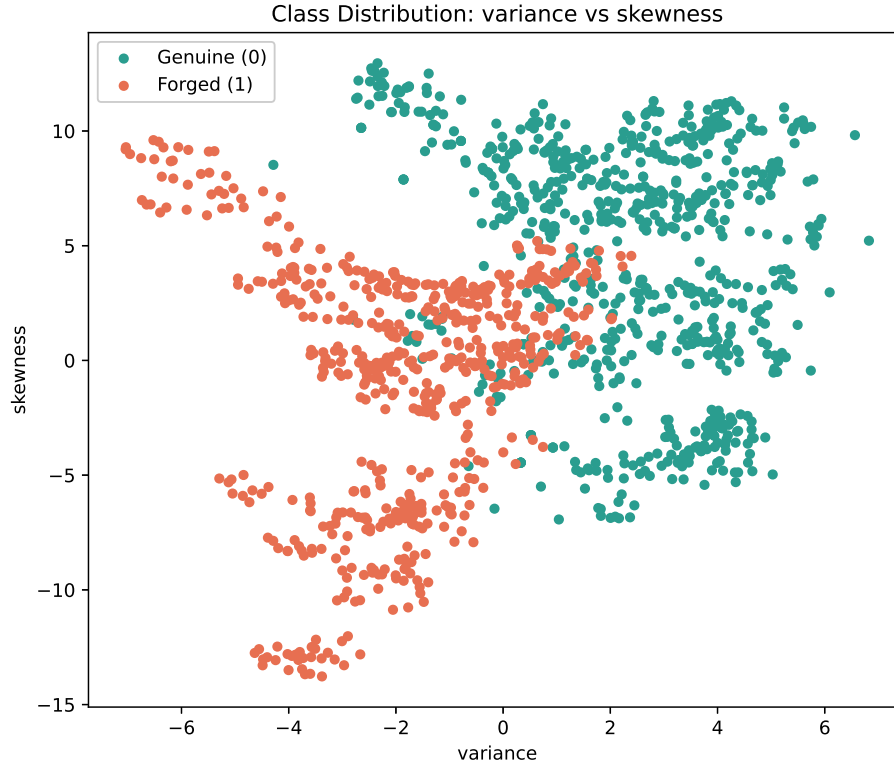


Figure 3: Scatter Plot showing class separability

Inference: From the Inference 1 and Inference 2 , we get variance and the skewness/curtosis as a important features ,therefore plotting a scatter plot against variance and skewness shows a almost distint sepeartion between both with some overlap near ≈ 0 . This confirms the dataset is nearly linearly sepearable.

iv) Feature Boxplots

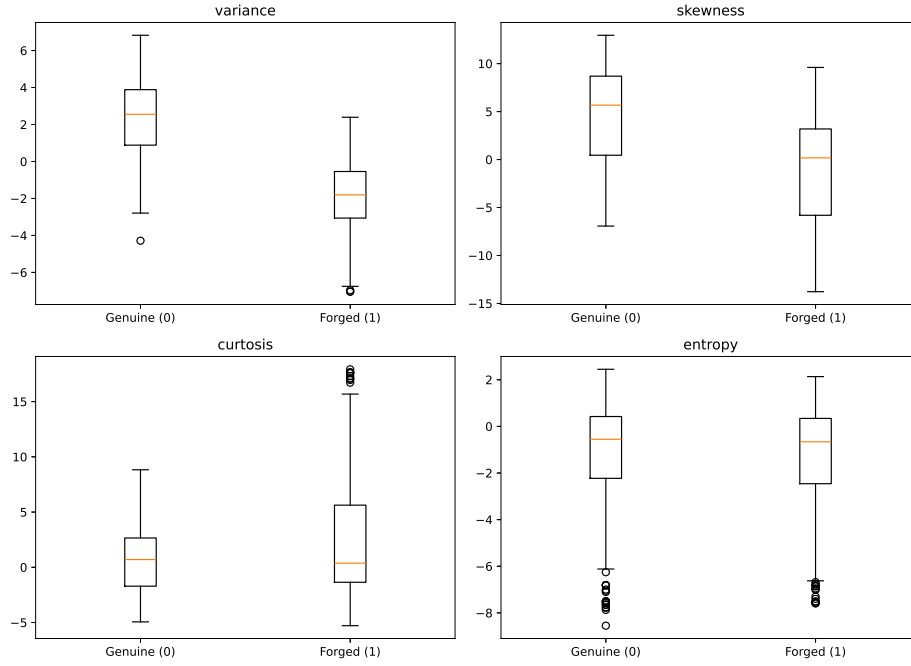


Figure 4: Boxplots for detecting feature spread and outliers

Inference: The variance shows no overlap between their medians while entropy shows a complete overlap between their medians showing entropy as one of the weakest feature and confirming variance as a strongest feature. Kurtosis shows a cluster of high-value outliers belonging almost exclusively to forged notes, indicating their behaviour in the histogram.

v) Training Error vs Epoch

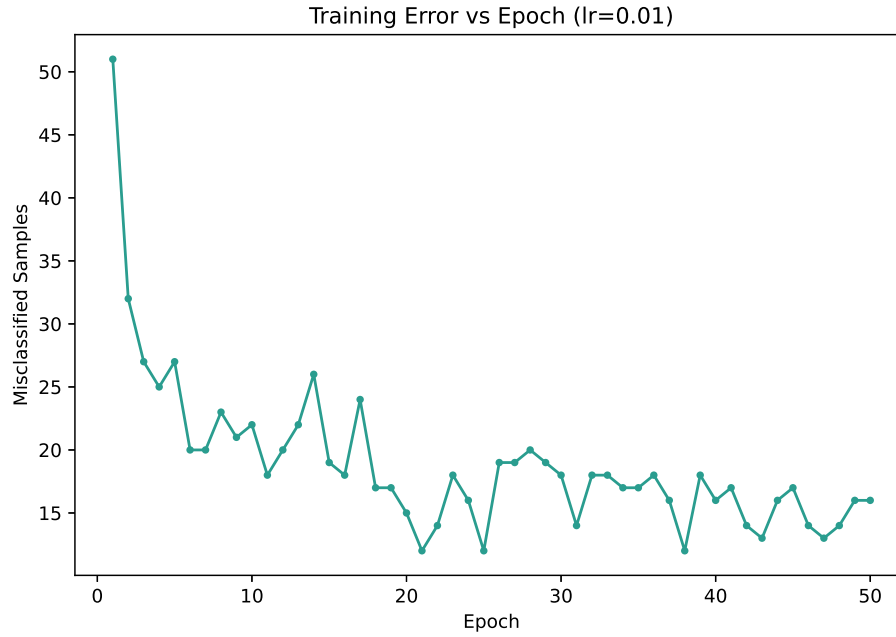


Figure 5: Training Error Variation over Epochs

Inference: Misclassified samples drop sharply from 51 to 12 within the first six samples and fluctuates between 12 to 20 indicating their sharp convergence at first but could not reach a perfect zero convergence indicating their behaviour in non-linearly-separable overlap region.

vi) Weight Evolution

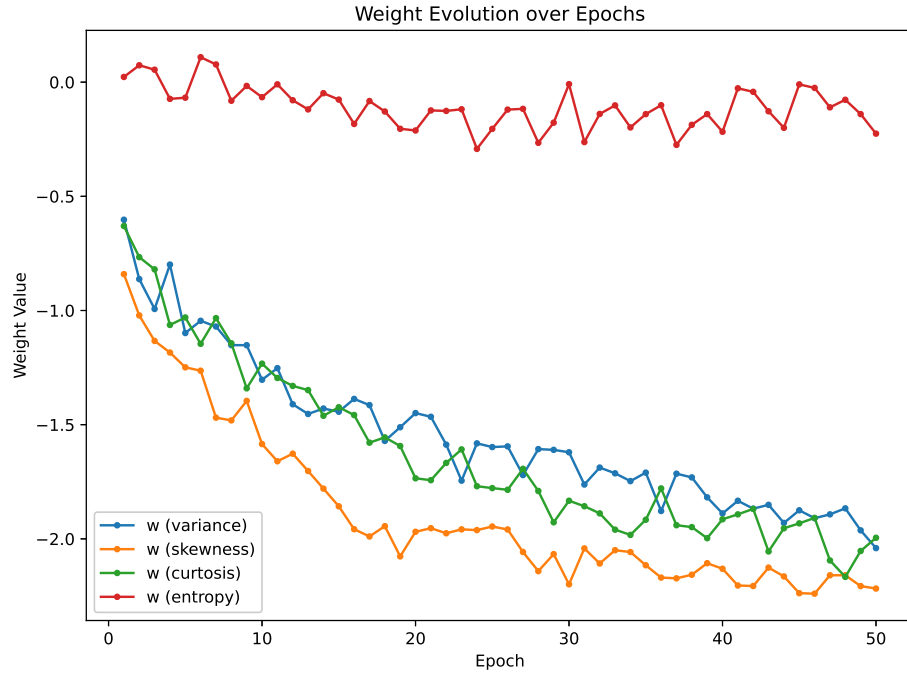


Figure 6: Evolution of Perceptron Weights during Training

Inference: The weights of variance, skewness and kurtosis shows a growth in the weights after epoch 20 while the weights of variance are standing near zero representing their weak feature importance observed in the above inferences and it is learnt to prioritize the other features.

vii) Bias Evolution

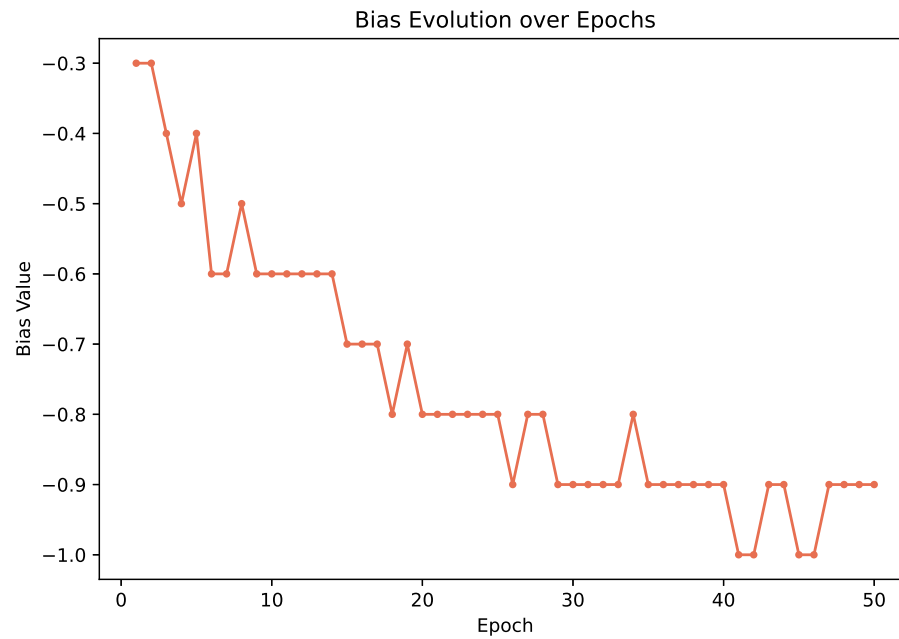


Figure 7: Evolution of Bias Value during Training

Inference: The bias decreases steadily from -0.03 to approximately -0.09 over the first 20 epochs and this shifts the hyperplane into a proper decision boundary.

viii) Confusion Matrix

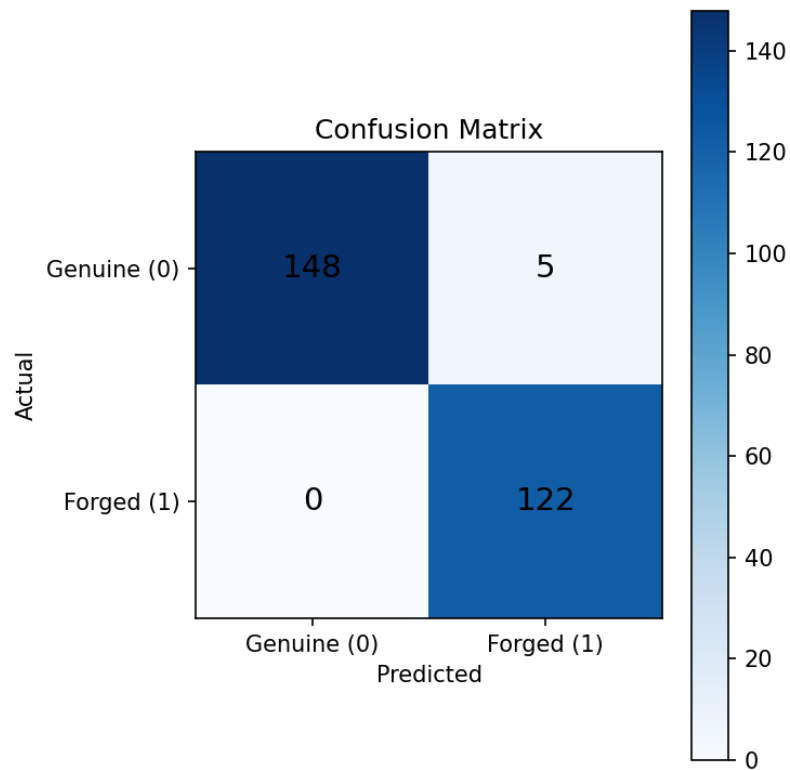


Figure 8: Confusion Matrix for Perceptron Classification Performance

Inference: The classifier achieved 148 true negatives, 122 true positives, 5 false positives, and 0 false negatives on the test set. The 0 FN indicates that every forged notes was identified as forged, which is one of the most important in an authentication system.

ix) Learning Rate Comparison

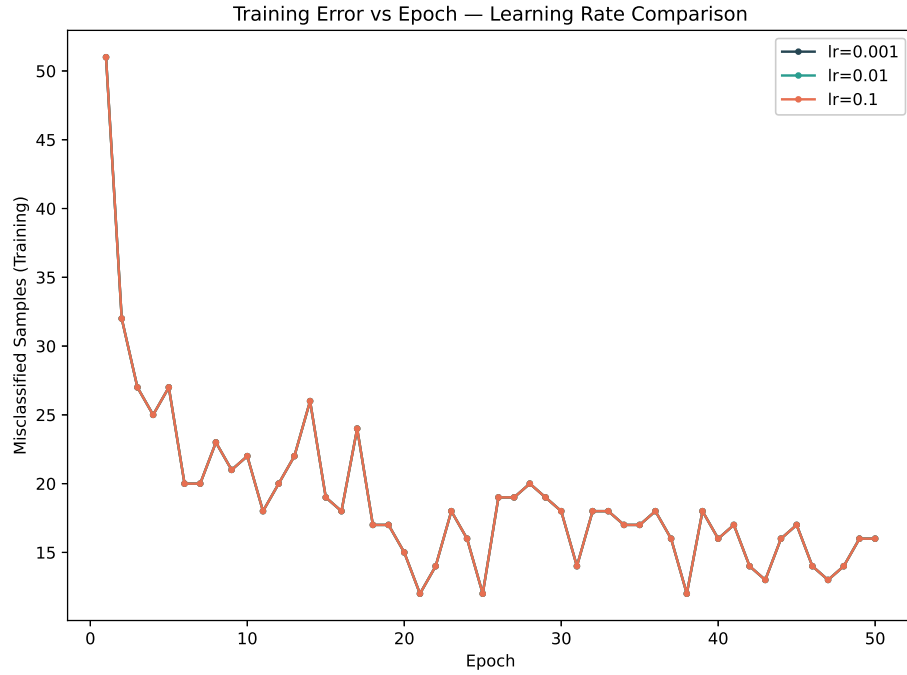


Figure 9: Comparison of Model Convergence for Different Learning Rates

Inference: All three learning rates (0.001, 0.01, 0.1) produced identical final classification performance and identical training error curves. This occurs because the decision boundary is completely dependent on the sign of the weighted sum and since we have initially represented weights as zero the weighted sum has grown proportionally equal in all learning rates.

x) Decision Boundary Visualization

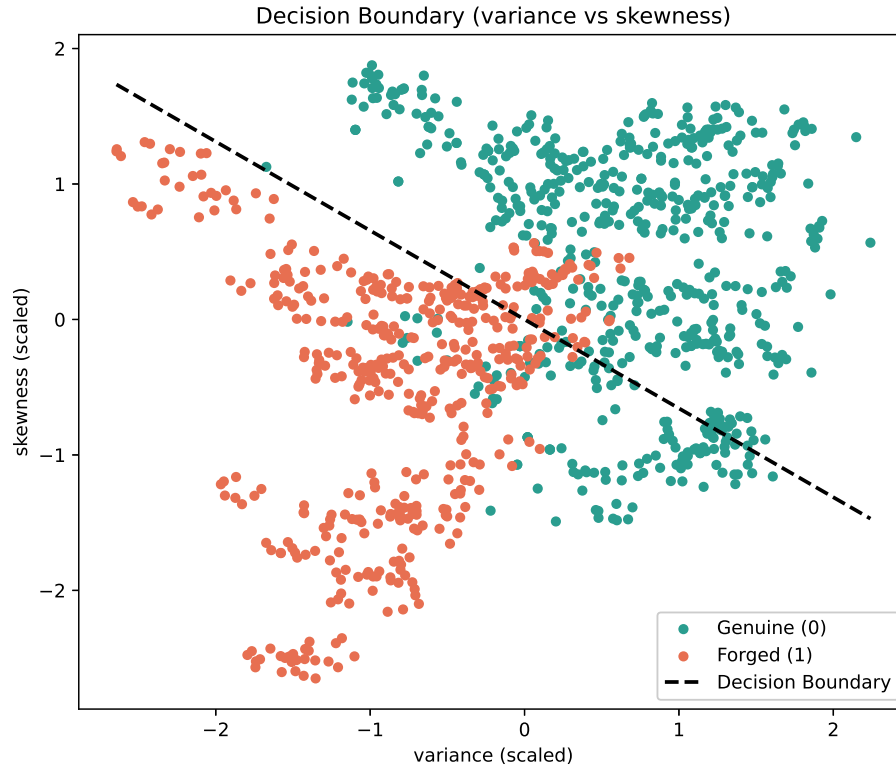


Figure 10: Decision Boundary obtained using Single Layer Perceptron

Inference: The learned boundary correctly separates the majority of genuine and forged notes when visualized using variance and skewness, with residual misclassifications concentrated in a narrow region surrounding the boundary line. This visually confirms the approximate linear separability of the dataset and explains the small number of test-set errors observed in the confusion matrix.

10. Performance Tables

Training Summary

Dataset Size	(1372, 5)
Train/Test Split	80 : 20
Learning Rate	0.01
Epochs	50
Final Weights	[-0.204, -0.2218, -0.1995, -0.0226]
Final Bias	-0.09
Accuracy	0.9818
Precision	0.9606
Recall	1.0000
F1-score	0.9799

Epoch-wise Learning

Epoch	Errors	Weight 1 (variance)	Weight 2 (skewness)	Bias
1	51	-0.060	-0.084	-0.030
5	27	-0.110	-0.125	-0.040
10	22	-0.130	-0.158	-0.060
20	15	-0.145	-0.197	-0.080
30	18	-0.162	-0.220	-0.090
40	16	-0.189	-0.213	-0.090
50	16	-0.204	-0.222	-0.090

9. Additional Tasks

1. **Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.**

Step gives a hard 0/1 jump at $z = 0$ with no usable slope anywhere, while sigmoid gives a smooth S-curve with a well-defined derivative everywhere. Since training relies on gradients to know how much to adjust each weight, a flat or undefined derivative gives no learning signal. This is exactly why our learning rate experiment showed identical results regardless of η , since step activation ignores how wrong a prediction was, only whether it was right or wrong. Modern networks use sigmoid, ReLU, etc. specifically so gradient descent has something to work with.

2. **Compare the implementation with Scikit-learn's Perceptron.**

Our from-scratch perceptron reached 98.18% accuracy and F1 of 0.9799, slightly outperforming sklearn's built-in `Perceptron` (96.73% accuracy, F1 of 0.9639) under the same learning rate and epochs. The final weight vectors differ somewhat between the two, likely due to internal differences in shuffling and stopping criteria. Both landing in a similar performance range is a good sanity check that our implementation is behaving correctly.

3. **Repeat the experiment using learning rates 0.001, 0.01 and 0.1.**

All three rates gave identical misclassification counts and identical final metrics. This happens because starting from zero weights makes the entire training trajectory simply scale with η , without changing the decision boundary's direction at all.

4. **Explain why a Single Layer Perceptron cannot solve the XOR problem.**

XOR's four points cannot be separated by any single straight line. the two positive-class points sit diagonally opposite the two negative-class points. Our perceptron predicted $[1, 1, 0, 0]$ against the true $[0, 1, 1, 0]$, getting every point wrong and never converging even after 20 epochs. This is exactly why multilayer perceptrons exist. stacking layers creates non-linear boundaries that a single-layer perceptron structurally cannot.

5. **Study the effect of feature normalization on model convergence.**

Without normalization, training reached 14 misclassified samples after 50 epochs; with normalization, 16. a smaller gap than expected. This is because our 4 features already have fairly similar raw ranges (11–27), so the usual scale-mismatch problem normalization fixes wasn't severe here. Normalization remains good practice generally, just less impactful on this particular dataset.

6. **Implement the Perceptron Learning Algorithm for the OR, AND and NOT logic gates, showing the weights after each update and plotting the decision boundary after each weight update.**

For each gate, a perceptron was trained from scratch using the same weight update rule as the main experiment: $w \leftarrow w + \eta(y - \hat{y})x$ and $b \leftarrow b + \eta(y - \hat{y})$, starting from $w = 0$, $b = 0$ with $\eta = 1$. Since OR, AND and NOT are all linearly separable functions, the perceptron converged to a perfectly separating boundary for every gate, unlike the banknote dataset where a small overlap region prevented full convergence.

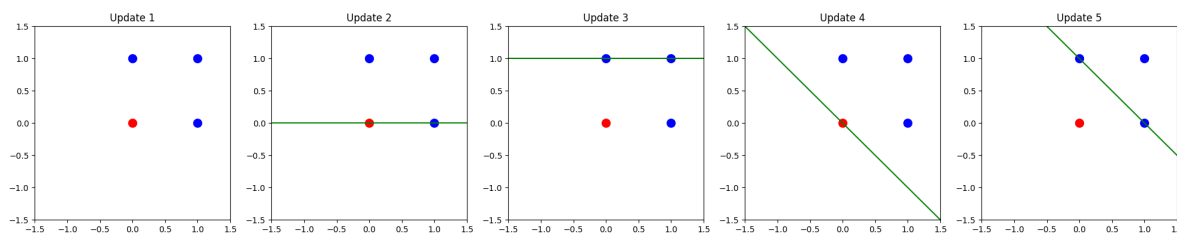


Figure 11: Decision boundary of the perceptron after each weight update while learning the OR gate

Inference: The perceptron needed 5 weight updates to converge on OR. The boundary line starts at the origin (all weights zero) and rotates and shifts across updates until it settles at $w = [1, 1]$, $b = -1$, which correctly places only (0,0) on the negative side and the remaining three points on the positive side.

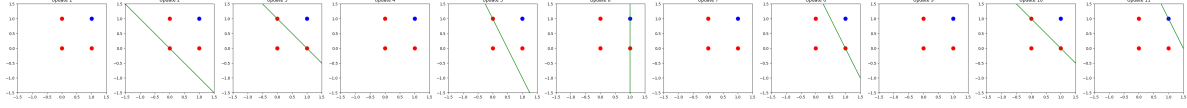


Figure 12: Decision boundary of the perceptron after each weight update while learning the AND gate

Inference: AND required considerably more updates than OR (18 updates across 6 epochs) before converging to $w = [2, 1]$, $b = -3$. The extra difficulty comes from AND having only one positive example, $(1, 1)$, out of four. early updates repeatedly overshoot and briefly misclassify points that were previously correct, before the boundary finally settles with only $(1, 1)$ on the positive side.

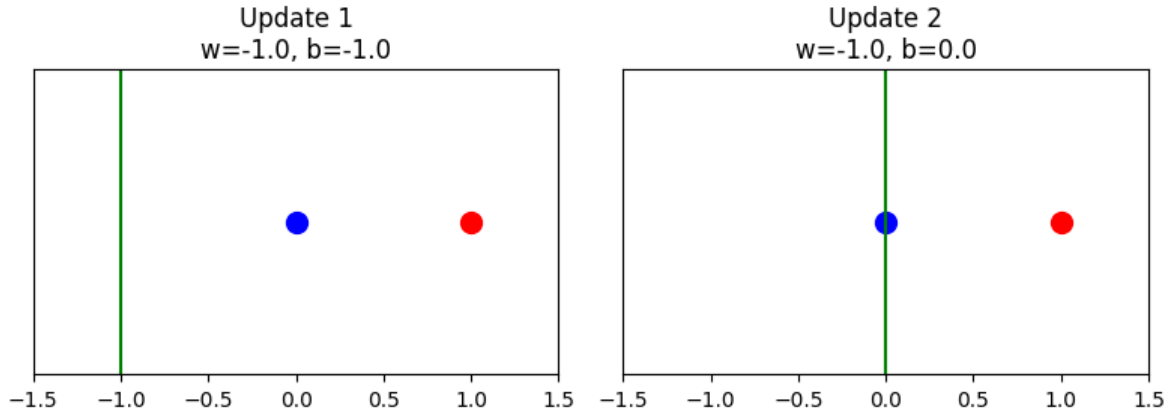


Figure 13: Decision point of the perceptron after each weight update while learning the NOT gate

Inference: Since NOT has a single input, its decision boundary collapses from a line to a single point on a number line, $x = -b/w$. The perceptron converged in just 2 updates to $w = -1$, $b = 0$, giving the boundary point $x = 0$: any input ≥ 0 falls on the negative-weight side and is pushed toward class 0 as x increases, correctly inverting $0 \mapsto 1$ and $1 \mapsto 0$.

11. Discussion

The experiment showed that a simple linear model can classify banknotes as authentic or forged with high accuracy, even without ever fully converging during training. The EDA stage turned out to be the most informative part of the whole process. the histograms, boxplots and correlation heatmap all independently pointed to variance as the strongest discriminative feature and entropy as the weakest, and this ranking was later confirmed by the perceptron's own learned weights, where the weight on entropy stayed close to zero throughout training. This agreement between EDA and model behaviour gives good confidence that the classifier is picking up genuine structure in the data rather than fitting noise.

The training error and decision boundary plots both point to the same underlying limitation: a small overlap region between the two classes prevents the perceptron from ever reaching zero

misclassifications. This is an expected consequence of using a hard step activation on data that is only approximately linearly separable, not a bug in the implementation. Despite this, the final test performance was strong, with perfect recall meaning no forged note went undetected, which is arguably the most important property for a banknote authentication system, since letting a forgery through is a far costlier mistake than flagging a genuine note by accident.

The learning rate experiment also revealed a subtle theoretical property of the vanilla perceptron that wasn't obvious going in. since predictions depend only on the sign of the weighted sum, and weights start at zero, changing η has no effect on the classification outcome at all, only on the magnitude of the final weights. Comparing against Scikit-learn's built-in Perceptron gave a useful sanity check too, since both implementations landed in a similar accuracy range, suggesting the from-scratch version is behaving correctly rather than just coincidentally producing reasonable-looking numbers.

12. Conclusion

In this experiment, a Single Layer Perceptron was implemented from scratch and trained on the Banknote Authentication dataset to classify notes as authentic or forged. The perceptron learning rule was used to update the weights and bias over 50 epochs, and the model achieved a test accuracy of 98.18%, with precision, recall and F1-score all above 0.96, including a perfect recall of 1.0. Even though the training error did not fully converge to zero and kept oscillating between roughly 12 and 20 misclassifications in the later epochs, the model still performed very well on unseen data. This suggests that the two classes are mostly, but not perfectly, linearly separable, which matches what was observed earlier in the scatter plots and the decision boundary visualization.

The learning rate comparison showed that changing η between 0.001, 0.01 and 0.1 did not affect the number of misclassifications per epoch, since the step activation only depends on the sign of the weighted sum and not its magnitude. Comparing the results with Scikit-learn's Perceptron implementation (96.73% accuracy) further confirmed that the from-scratch implementation is working as intended, since both models gave comparable performance on the same data.

Overall, this experiment helped build a concrete understanding of how a single artificial neuron learns through repeated weight updates, and also made the limitations of a basic perceptron with a step activation function clear when the underlying data isn't perfectly linearly separable. This naturally motivates the need for more capable models like Multilayer Perceptrons, which will be explored in later experiments.

13. Report Contents

The report should include:

- Objective
- Background Theory
- Activation Functions
- Dataset Description
- Experimental Procedure
- Source Code

- Experimental Results
- Generated Plots with Interpretation
- Performance Tables
- Discussion
- Conclusion
- References

14. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository. Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.